

Dynamic memory allocation

C programming · CM10 · 90 minutes

ISTIC · University of Rennes 2026-2027



\$ whoami

Not a C expert ... : ')

Louis Ledoux

Associate Professor

University of Rennes · ISTIC

IRISA / Inria · Taran

▶ Arithmetic circuits: **MLIR (LLVM)**
/ **CIRCT**.

▶ **Systolic arrays** and small
floating-point formats.

Previously: Inria / INSA Lyon,
Emeraude.

PhD: **BSC** / **UPC**.

bynaryman.github.io



MareNostrum 4 · BSC,
Barcelona

Lucia Meler / Martidaniel · CC BY-SA 4.0

Today

1. Allocate an array.
2. Find and fix memory errors.
3. Arrays, strings and matrices.
4. User-defined types.

1. Allocate an array

How much space do we need?

Four integers, then one hundred?

A program has room for **four integers**.

The user asks to store **100**. What must change?

`demos/allocation/01-fixed-array.c`

Terminal: `make run FILE=01-fixed-array.c`

Does reading a count create more space?

Predict what happens when we enter **100**.

```
int count;
int p[4];
if (scanf("%d", &count) != 1) return EXIT_FAILURE;
if (count <= 0 || count > 4) {
    fputs("Choose 1 to 4: the array has four elements.\n", stderr);
    return EXIT_FAILURE;
}
```

demos/allocation/02-capacity.c

Terminal: make run FILE=02-capacity.c

Choose the capacity during execution

How can we reserve **count integers**?

```
int *p = malloc(count * sizeof(int));  
if (p == NULL) {  
    fputs("Allocation failed\n", stderr);  
    return EXIT_FAILURE;  
}
```

demos/allocation/03-missing-free.c

Terminal: make run FILE=03-missing-free.c

Count elements, request bytes

```
void *malloc(size_t size);    // From <stdlib.h>  
int *p = malloc(100 * sizeof(int));
```

- ▶ `size_t` is unsigned, with at least **16 value bits**.
- ▶ `sizeof` is an **operator** giving a size in bytes.
- ▶ `sizeof(int)` is the size of one `int`, not the pointer.

Sizes depend on the platform. In C, the `void *` result needs no cast.

2. Find and fix memory errors

The program prints a result. Is it correct?

It prints correctly. Is anything wrong?

Run the allocated version with **100**, then inspect it:

```
make run FILE=03-missing-free.c  
make valgrind FILE=03-missing-free.c
```

Where was the unreleased block allocated?

Read first, then release

The array has been allocated and filled with zeros.

```
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

```
free(p);
```

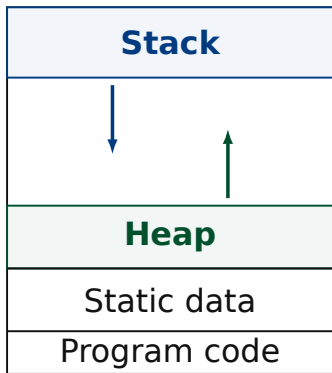
free(p) releases the
block **after its last read**.

**All heap blocks
freed**

Valgrind now reports
no leak.

Where does the program keep its data?

0xffff



0x0000

count and pointer p
Automatic locals

Room to grow
Opposing directions

p[0] to p[count-1]
Allocated integers

Globals and static

Illustrative 16-bit address space; real address widths and layouts vary.

What will the array contain?

malloc succeeded. We removed the loop that wrote zeros.

```
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

Uninitialised value

Valgrind reports that
printf uses values we
never wrote.

malloc reserves storage; it **does not initialise the integers**.

Write before reading

Restore the loop **before** printf.

```
for (int i = 0; i < count; ++i)
    p[i] = 0;

printf("first=%d, last=%d\n",
      p[0], p[count - 1]);
```

Every element now has a value before it is read.

Zero errors
Rerun Valgrind to
check this execution.

Four integers: which write is outside?

`count = 4`. The allocation holds `p[0]` through `p[3]`.

```
for (int i = 0; i <= count; ++i)
    p[i] = 0;
```

Invalid write

`<=` also allows `i = 4`.

There is **no element** `p[4]`
in this allocation.

Use `i < count`: four writes, with indices **0, 1, 2, 3**.

Can we read after free?

p points to four integers initialised to zero.

```
free(p);  
  
printf("first=%d, last=%d\n",  
      p[0], p[count - 1]);
```

After free(p), **p is a dangling pointer.**

Invalid read

Valgrind points to printf:
the block was **already
freed.**

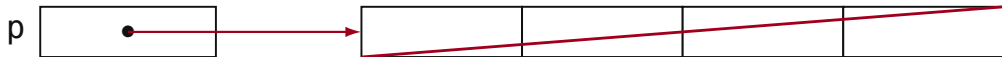
Dangling pointer

A pointer to an object **whose lifetime has ended**.

`free(p)` releases the block; it **does not set p to NULL**.

Pointer

Released block



Do not access these elements

Repair: finish reading before releasing the block.

```
printf("first=%d, last=%d\n", p[0], p[count - 1]);  
free(p);
```

Does setting p to NULL free the block?

p is the **only pointer** to the allocated block.

```
p = NULL;  
free(p);
```

free(NULL) does nothing.
We lost the address: a
memory leak.

Pointer

p	NULL
---	------

Allocated block

0	0	0	0
---	---	---	---

Address lost; block still allocated

Local objects can expire too

```
int *bad_result(void) {  
    int value = 42;  
    return &value;  // The local dies when this call ends.  
}
```

A dangling pointer need not come from free

Returning an address does not extend an automatic object's lifetime.

Who controls an object's lifetime?

Static storage

Globals and static objects last for the **whole program**, with storage reserved before execution.

Automatic storage

Locals last for their **block execution**. Recursive calls have separate locals.

Storage is managed automatically. Deep recursion can **exhaust the stack**.

Who is responsible for reclamation?

C: explicit responsibility

For every successful allocation, decide who will release it and when.
Repeated leaks consume more memory.

Java: garbage collection

The runtime can reclaim unreachable objects automatically.

Four functions in `<stdlib.h>`

Function	Purpose
<code>malloc(bytes)</code>	Allocate uninitialised storage
<code>calloc(count, size)</code>	Allocate storage with all bits zero
<code>realloc(p, bytes)</code>	Resize an allocation; it may move
<code>free(p)</code>	Release an allocation

The free contract

```
void free(void *pointer);    // Declared in <stdlib.h>
```

- ▶ Pass the allocation's **start address**, once.
- ▶ `free(NULL)` does nothing; `free` returns no value.
- ▶ An invalid or repeated release is **undefined behaviour**.

Checkpoint · questions 1-5

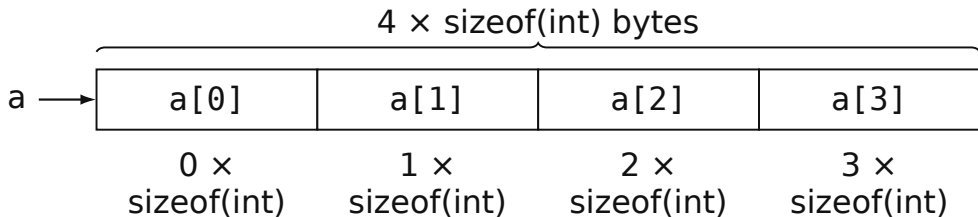
Answer questions **1-5**, then discuss your reasoning.
Open the allocation quiz.

3. Arrays, strings and matrices

Allocate data, then locate each element.

A one-dimensional integer array

```
int *a = malloc(4 * sizeof(int));
```



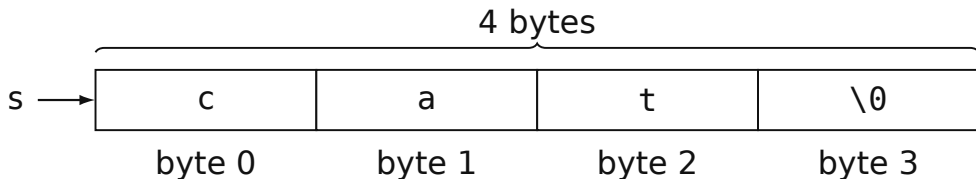
Use `a[i]`, equivalently `*(a + i)`, for **$0 \leq i < 4$** .

`demos/allocation/09-array.c`

Terminal: `make run FILE=09-array.c`

A character array can hold a string

```
char *s = malloc(4 * sizeof(char));
```



Three letters need **four bytes**, including '`\0`'.

`demos/allocation/10-string.c`

Terminal: `make run FILE=10-string.c`

Assigning a pointer is not copying text

Predict the values of p and s, and the fate of the allocation.

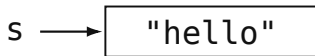
```
char *s = malloc(100);  
if (!s) return EXIT_FAILURE;  
s = "hello";  
char *p = s;  
s = "bye";
```

demos/allocation/11-string-assignment.c

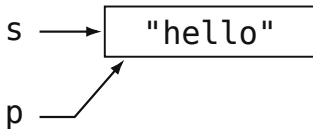
Terminal: make run FILE=11-string-assignment.c

Follow the three assignments

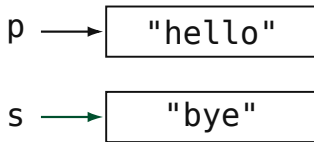
`s = "hello"`



`p = s`



`s = "bye"`



Original heap block: 100 bytes, now unreachable

The literals are separate from the lost heap block. Neither pointer can now be passed to `free`.

Copy characters into owned storage

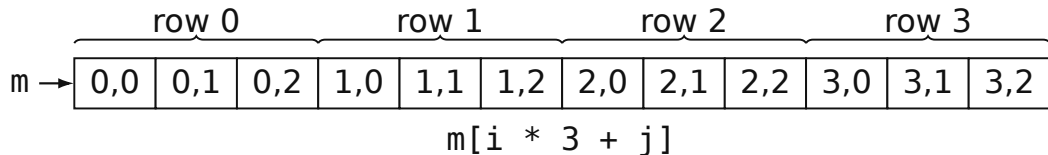
```
char *s = malloc(6);  
if (!s) return EXIT_FAILURE;  
strcpy(s, "hello"); // Include <string.h>.  
printf("%s\n", s);  
free(s);
```

Alternatively, just reference a literal

Use `const char *s = "hello";` when no writable copy is needed.

A 2D array in one block

```
int *m = malloc(4 * 3 * sizeof(int));  
/* After checking m: element (i, j) is m[i * 3 + j]. */
```



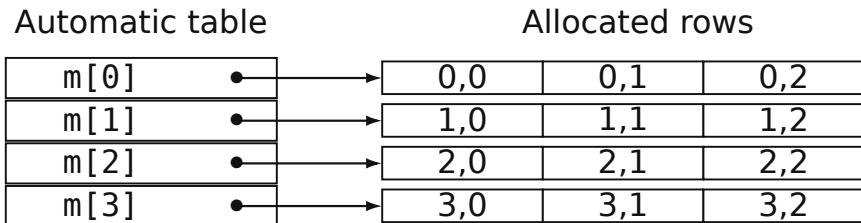
One allocation, one **free(m)**.

demos/allocation/12-matrix-flat.c

Terminal: make run FILE=12-matrix-flat.c

An array of row pointers

```
int *m[4]; // Automatic table of four pointers.  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```



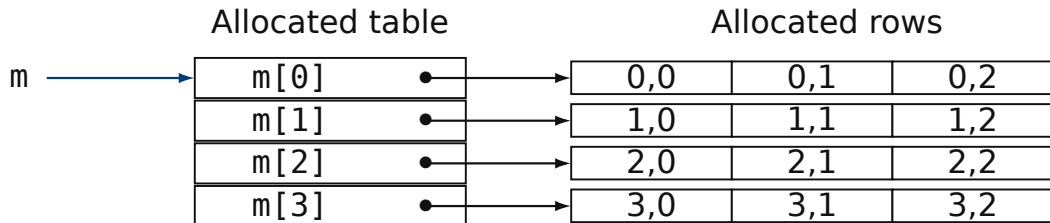
Check each row. Use `m[i][j]`; free each row, **not** `m`.

`demos/allocation/13-matrix-rows.c`

Terminal: `make run FILE=13-matrix-rows.c`

Allocate the pointer table too

```
int **m = malloc(4 * sizeof(int *));  
for (int i = 0; i < 4; ++i)  
    m[i] = malloc(3 * sizeof(int));
```



Check `m` first, then each row. Free **rows first**, then `free(m)`.

`demos/allocation/14-matrix-dynamic.c`

Terminal: `make run FILE=14-matrix-dynamic.c`

Checkpoint · questions 6-8

Answer questions **6-8**, then discuss your reasoning.
Open the allocation quiz.

4. User-defined types

Group data and access its members.

We want to keep a street name and a house number in one object.

Three tools

- ▶ struct: group the street and number.
- ▶ enum: name integer constants.
- ▶ typedef: give a type another name.

`demos/allocation/15-types.c`

Terminal: `make run FILE=15-types.c`

Enumerations

```
enum day { mon, tue, wed, thu, fri, sat, sun };  
enum day today;  
today = wed;
```

By default, values start at **0** and increase by **1**.

Named integer constants

mon is 0; wed is 2; sun is 6.

A structure groups several members

```
struct address {  
    char street[100];  
    int number;  
};
```

A type definition

Members may be integers, floating-point values, arrays, pointers or other structures. No address object exists yet.

Define a type, then create objects

```
struct address home;  
struct address office;
```

These definitions allocate **two distinct objects** of the same type.

Local objects need initialisation

Ordinary local members have indeterminate values until initialised or assigned. The type definition alone sets no values.

Initialising a structure

```
struct address home = { "Paul Bert", 12 };
```

Or combine the type and object definitions:

```
struct address {  
    char street[100];  
    int number;  
} home = { "Paul Bert", 12 };
```

Same members, different order

```
struct grouped {  
    int id1, id2;  
    char c1, c2;  
    float ratio;  
};  
struct interleaved {  
    int id1; char c1;  
    int id2; char c2;  
    float ratio;  
};
```

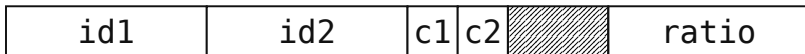
Padding

The implementation may insert unused bytes between members and at the end to satisfy alignment requirements.

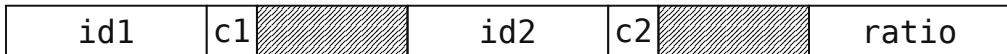
- ▶ Members appear in declaration order.
- ▶ `sizeof(struct T)` includes padding.
- ▶ Layout and alignment depend on the implementation.

See where the padding goes

Grouped: 16 bytes



Interleaved: 20 bytes



Here: 4-byte int/float, alignment 4; 1-byte char.

16 vs 20 bytes in this layout. Measure with `sizeof` and `offsetof`.

`demos/allocation/16-padding.c`

Terminal: `make run FILE=16-padding.c`

Access members through an object

```
struct address home = { "Paul Bert", 12 };  
strcpy(home.street, "Rue de Paris");  
home.number = 14;
```

The dot operator

Use `.` with a structure object.

`strcpy` needs `<string.h>` and enough space for the terminator.

Access members through a pointer

```
struct address *p = &home;  
strcpy(p->street, "Rue de Paris");  
p->number = 16;
```

The arrow operator

Use `->` with a pointer. `p->number` is the same member as `(*p).number`.

Both forms above modify **home**.

typedef: a name for a type

```
typedef int LENGTH;  
typedef enum { FALSE, TRUE } BOOLEAN;  
typedef struct date DATE;
```

Alias

A typedef introduces a type name, not an object or a distinct underlying type. Then write `LENGTH width;`.

Combine a date and a day of the week

```
struct date { short day, month; int year; };  
typedef enum { mon, tue, wed, thu, fri, sat, sun } DAY;  
typedef struct { DAY day; struct date date; } A_DATE;
```

A nested structure

A_DATE contains a DAY and a complete struct date.

Access nested members

```
A_DATE today = { wed, { 3, 9, 2014 } };  
today.day = wed;  
today.date.year = 2014;  
A_DATE *p = &today;  
p->day = today.day;  
p->date.year = 2026;
```

p->date.year reaches the nested member of **the same object**.

demos/allocation/17-nested.c

Terminal: make run FILE=17-nested.c

Checkpoint · questions 9-12

Answer questions **9-12**, then discuss your reasoning.
Open the allocation quiz.